Kubernetes Agent Service

- Service Overview
- **Alerts**: https://alerts.gitlab.net/#/alerts?filter=%7Btype%3D%22kas%22%2C%20tier%3D%22sv%22%7D
- **Label**: gitlab-com/gl-infra/production~"Service::KAS"

Logging

- kas

Summary

The GitLab Agent for Kubernetes (`agentk`) is an active in-cluster component for solving GitLab and Kubernetes integration tasks in a secure and cloud-native way. It enables:

- Integrating GitLab with a Kubernetes cluster behind a firewall or NAT (network address translation).
- Allows Gitlab Real-time access to the Kubernetes API endpoints in a users cluster
- Grants Gitlab the ability to build extra functionality on top of the pieces above, e.g. Kubernetes network security alerts
- Other features.

More information can be found at https://docs.gitlab.com/ee/user/clusters/agent/

GitLab Kubernetes Agent Server (`kas`) is the server component that runs along GitLab.

Architecture

```
top to bottom direction
skinparam sequenceMessageAlign left
skinparam roundcorner 20
skinparam shadowing false
skinparam rectangle {
  BorderColor DarkSlateGray
}

card "Gitlab User Kubernetes Cluster" as GUKC {

  rectangle "agentk Pod" as AGENTK {
  }

}
```

```
cloud "Internet" as INTERNET {

}

card "kas.gitlab.com GCP Load Balancer" as LB {
}

rectangle "GKE Regional Cluster" as GKE {
  card "gitlab namespace" as GPRD {
    rectangle "KAS Pod" as KAS
  }

}
rectangle "Virtual Machines" as VMS {
  rectangle "GitLab.com /api" as GLAPI
  rectangle "Gitaly" as GITALY
  rectangle "redis" as REDIS
}


AGENTK -- INTERNET
INTERNET --> LB
LB --> KAS
KAS --> GLAPI : Authn/Authz of agentk
KAS --> GITALY : Fetch data from git repo
KAS --> REDIS: Store/Read info about `agentk` connections
```

Dependencies

1. GCP HTTPS Load Balancer, is used to load balance requests between the agentk (and the internet) and kas.
2. GitLab Web (Rails) server, which serves the internal API for kas.
3. Gitaly, which provides repository blobs for the agent configuration, and K8s resources to be synced.
4. Redis, which is used to store:
   - Information about `agentk` access tokens to allow us to do rate limiting against `kas` per token.
   - Tracking connected `agentk` agents to kas.
   - Other information.

Agent, KAS, and Rails Architecture

See https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/blob/master/doc/architecture.md#high-level-architecture

We have two components for the Kubernetes agent:

- The GitLab Kubernetes Agent Server (`kas`). This is deployed server-side together with the GitLab web (Rails), and Gitaly. It's responsible for:
  - Accepting requests from `agentk`.
  - Authentication of requests from `agentk` by querying `GitLab RoR`.
  - Fetching agent's configuration from a corresponding Git repository by querying `Gitaly`.
  - Agent configuration-dependent tasks (features).
- The GitLab Kubernetes Agent (`agentk`). This is deployed to the user's Kubernetes cluster. It is responsible for:
  - Keeping a connection established to a `kas` instance
  - Agent configuration-dependent tasks (features).

Performance

A rate limit on a per-client basis can be configured with the `agent.listen.connections_per_token_per_minu` setting - the default is 40,000 new connections per minute per agent. This requires Redis in order to track connections per agent. This rate limiting was introduced in https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/merge_requests/103.

The frequency of gRPC calls from `kas` to `Gitaly` can be configured too. See defaults in https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/blob/master/pkg/kascfg/kascfg_defaults.yaml.

Scalability

1. The `kas` chart is configured by default to autoscale by using a HorizontalPodAutoscaler. The `HorizontalPodAutoscaler` is configured to target an average value of 100m CPU. It will initially default to two pods, with the ability to scale up to a maximum of ten. Production configuration can be seen in `gprd.yaml.gotmpl`.
2. The current implementation of the liveness check simply returns a HTTP 200 OK, so is only reliable for basic determination of a pods health. The chart configuration uses basic HTTP GET for readiness and liveness checks.

Availability

Durability

`kas` uses Redis for caching and cross-replica information exchange. In Gitlab.com this is the main redis cluster.

Security/Compliance

An initial security review was done at https://gitlab.com/gitlab-com/gl-security/appsec/appsec-reviews/-/issues/30 and the summary is as follows

1. The team audited the `gitlab-agent` codebase from the `kas` part of the source code. They also audited the `agentk` to local cluster communication, and `agentk` to `kas` communication.
2. The team noted "The data flow within kas makes a good impression with respect to security practices. The only information which comes from the agent is the agent token. All other information is pulled from the GitLab API. This helps a lot to avoid logic errors and bypasses based on input from the agent."
3. While currently every agent uses a generated token to authenticate itself to Gitlab, further expansion is needed on the authentication and authorization model of `kas` in order to better control which agent has access to which repositories (inside the users permissions structure). This is being tracked in https://gitlab.com/gitlab-org/gitlab/-/issues/220912

Monitoring/Alerting

Kibana

Select the pubsub-kas-inf-gprd-*index pattern. (pubsub-kas-inf-gstg-* for staging)

staging: https://nonprod-log.gitlab.net/goto/9f205372ad310869528fc2cb5336baff

production: https://log.gprd.gitlab.net/goto/33a5e2d548b67b2247de5aa8169c47e8

Grafana Dashboards

https://dashboards.gitlab.net/dashboards/f/kas/kas-kubernetes-agent-server

Sentry

https://new-sentry.gitlab.net/organizations/gitlab/issues/?project=11

Tracing

https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/tracing

Links to further Documentation

- Kubernetes Agent Readiness Review